

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN

Modalidad a distancia

Trabajo Práctico Integrador

Gestión de Datos de Países en Python: filtros, ordenamientos y estadísticas

Materia: Programación 1

Proyecto: Gestión de Datos de Países en Python

Estudiantes: Josías Alexis Avram

Noelia Maricel Rodas

Fecha de entrega: 15 de junio de 2026

Índice

Índice	2
1. Introducción y objetivo	3
2. Marco teórico	3
2.1. Listas	3
2.2. Diccionarios	3
2.3. Funciones	3
2.4. Estructuras condicionales	3
2.5. Estructuras repetitivas	4
2.6. Ordenamientos	4
2.7. Estadísticas básicas	4
2.8. Archivos CSV	4
3. Decisiones técnicas y arquitectura	5
3.1. Estructura de datos elegida	5
3.2. Modularización en dos módulos	5
3.3. Manejo de errores y validaciones	5
3.4. Diagrama de flujo del programa principal	5
3.5. Capturas de pantalla de la ejecución	6
4. Dificultades y conclusiones	7
4.1. Dificultades encontradas	7
4.2. Conclusiones	7
5. Bibliografía y webgrafía	7
6. Enlaces de la entrega	7

1. Introducción y objetivo

El presente informe documenta el desarrollo del Trabajo Práctico Integrador de la materia Programación 1, cuyo objetivo es construir una aplicación de consola en Python para gestionar información sobre países. El sistema lee los datos desde un archivo CSV y permite agregar y actualizar países, buscarlos por nombre, filtrarlos por distintos criterios, ordenarlos y calcular estadísticas. El trabajo integra y afianza los conceptos centrales de la asignatura: listas, diccionarios, funciones, estructuras condicionales y repetitivas, ordenamientos, estadísticas básicas y manejo de archivos.

Cada país se modela como un diccionario con las claves **nombre**, **poblacion**, **superficie** y **continente**, y el conjunto completo se almacena en una lista de diccionarios, estructura que organiza los datos de forma clara y eficiente.

2. Marco teórico

A continuación se explican los conceptos de Programación 1 aplicados en el proyecto, vinculándolos con su uso concreto dentro del sistema.

2.1. Listas

Una lista es una estructura de datos ordenada y mutable que permite almacenar una colección de elementos bajo un mismo nombre, accesibles por su índice. En este proyecto la lista es la estructura principal: contiene todos los países cargados y sobre ella se realizan los recorridos para buscar, filtrar, ordenar y calcular estadísticas. Su carácter mutable permite agregar nuevos países con **append()** sin recrear la colección.

2.2. Diccionarios

Un diccionario almacena pares clave-valor, lo que permite acceder a cada dato por un nombre descriptivo en lugar de una posición numérica. Cada país se representa como un diccionario (**pais['poblacion']** es más legible y seguro que **pais[1]**). Esta elección evita el problema de las listas paralelas, donde mantener varios listados sincronizados por índice es propenso a errores y a la pérdida de datos.

2.3. Funciones

Las funciones encapsulan bloques de código reutilizables, idealmente con una única responsabilidad. El sistema está modularizado en funciones pequeñas y específicas (**leer_csv**, **agregar_pais**, **filtrar_por_continente**, **ordenar_paises**, **mostrar_estadisticas**, entre otras), lo que mejora la legibilidad, facilita las pruebas y evita la duplicación de código.

2.4. Estructuras condicionales

Las condicionales (**if**, **elif**, **else**) permiten que el programa tome decisiones según se cumplan o no ciertas condiciones. Se usan en todo el sistema: para interpretar la opción elegida en el menú, validar entradas, controlar rangos y decidir si una búsqueda arroja resultados.

2.5. Estructuras repetitivas

Los ciclos (**while** y **for**) ejecutan instrucciones de forma repetida. El ciclo **while** mantiene activo el menú principal hasta que el usuario decide salir y sostiene la revalidación de entradas inválidas; el ciclo **for** recorre la lista de países para mostrar, filtrar o calcular indicadores.

2.6. Ordenamientos

Ordenar consiste en reorganizar los elementos según un criterio. Se emplea la función **sorted()** junto con una función clave (**key**) que indica el campo a comparar (nombre, población o superficie) y el parámetro **reverse** para alternar entre orden ascendente y descendente. **sorted()** devuelve una lista nueva, por lo que no altera el orden original de los datos.

2.7. Estadísticas básicas

Las estadísticas resumen información de un conjunto de datos. El sistema calcula el país con mayor y menor población (con **max()** y **min()**), los promedios de población y superficie (sumatoria dividida por la cantidad de elementos) y el conteo de países por continente, acumulando en un diccionario auxiliar.

2.8. Archivos CSV

Un archivo CSV (valores separados por comas) es un formato de texto plano para almacenar datos tabulares. Mediante el módulo estándar **csv** y los gestores de contexto (**with open**) el programa lee el dataset al iniciar y persiste los cambios al guardar. Durante la lectura se realiza la conversión de tipos (texto a entero) y el manejo de excepciones para tolerar filas mal formadas o la ausencia del archivo.

3. Decisiones técnicas y arquitectura

3.1. Estructura de datos elegida

Se optó por una **lista de diccionarios**. El diccionario describe cada país con claves legibles y la lista agrupa el conjunto. Frente a la alternativa de listas paralelas, esta estructura mantiene la integridad de los datos (cada registro viaja completo) y simplifica el código de búsqueda, filtrado y ordenamiento.

3.2. Modularización en dos módulos

El proyecto se dividió en dos módulos con responsabilidades separadas, más un archivo integrador, lo que permitió que cada integrante trabajara su parte de forma independiente y la subiera a su propia rama de Git:

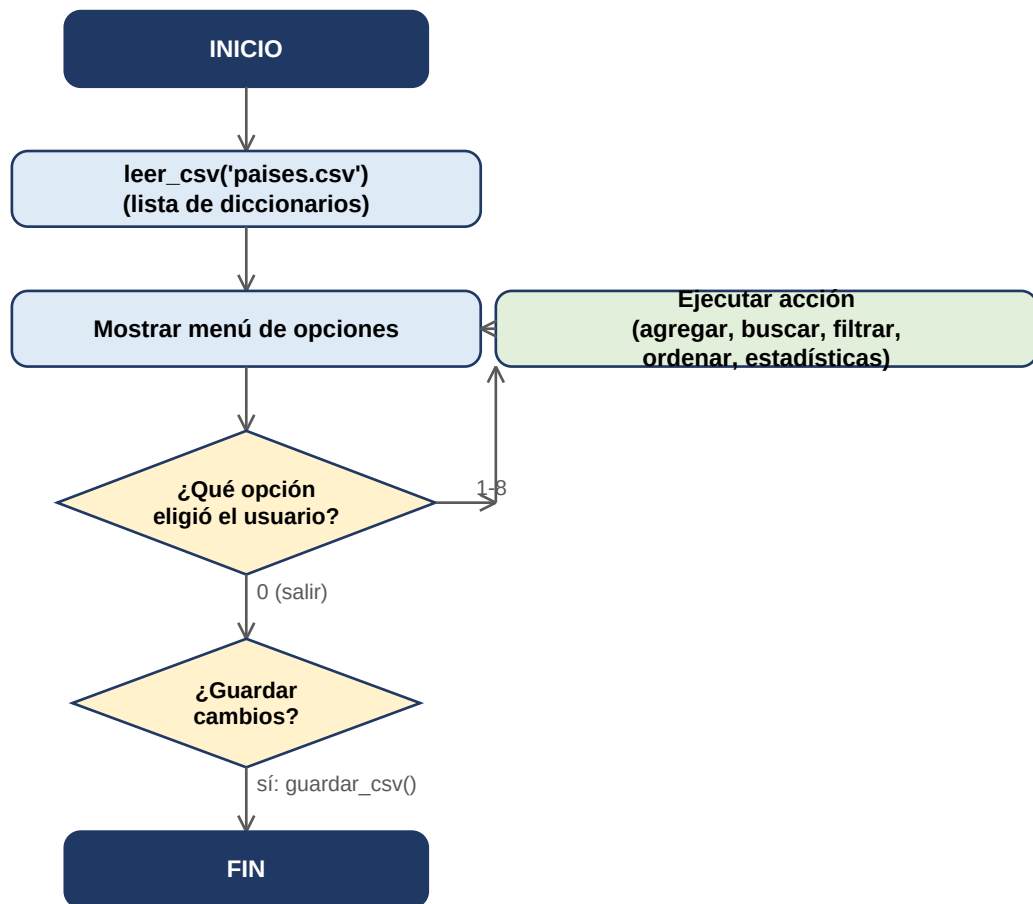
Archivo	Responsable	Contenido
core_datos.py	Noelia M. Rodas	Lectura/escritura del CSV, conversión de tipos y manejo de excepciones, estructura base, alta y actualización de países, validaciones.
consultas_reportes.py	Josías A. Avram	Búsqueda por nombre, filtros, ordenamientos (asc/desc) y estadísticas.
main.py	Integración	Menú de consola que conecta ambos módulos.

3.3. Manejo de errores y validaciones

Se aplicaron buenas prácticas reforzadas a partir de las devoluciones previas de la cátedra: uso de **try/except** en todas las conversiones numéricas para evitar caídas ante entradas no válidas; control de existencia del archivo antes de abrirlo; validación de campos vacíos y de países duplicados; y mensajes de éxito y error claros y consistentes.

3.4. Diagrama de flujo del programa principal

El siguiente diagrama representa el flujo general del sistema: carga inicial de datos, presentación del menú, ejecución de la opción elegida en bucle y guardado opcional al salir.



3.5. Capturas de pantalla de la ejecución

[Espacio reservado para insertar las capturas de pantalla de la ejecución del programa: menú principal, una búsqueda, un filtro, un ordenamiento y la pantalla de estadísticas.]

4. Dificultades y conclusiones

4.1. Dificultades encontradas

Durante el desarrollo, los principales desafíos fueron: la **conversión y validación de tipos** al leer el CSV, ya que una fila mal formada podía interrumpir todo el programa, lo que se resolvió validando cada fila e ignorando con aviso las incorrectas; el **manejo de acentos y codificación**, solucionado abriendo los archivos en UTF-8; y la **coordinación del trabajo en equipo con Git**, que se ordenó definiendo desde el inicio la interfaz de funciones y separando el código en módulos, de modo que cada integrante trabajara su archivo en su propia rama sin generar conflictos.

4.2. Conclusiones

El proyecto permitió integrar de manera práctica los contenidos de Programación 1. La elección de una lista de diccionarios demostró ser adecuada para mantener la integridad de los datos, y la modularización en funciones de única responsabilidad facilitó el desarrollo, la lectura y la corrección del código. Se reforzó especialmente el manejo de archivos y el control de errores, aspectos clave para construir programas robustos. El trabajo en equipo con control de versiones aportó una experiencia cercana al desarrollo profesional de software.

5. Bibliografía y webgrafía

- Python Software Foundation. *The Python Tutorial - Data Structures* (listas y diccionarios). <https://docs.python.org/es/3/tutorial/datastructures.html>
- Python Software Foundation. *csv - CSV File Reading and Writing*. <https://docs.python.org/es/3/library/csv.html>
- Python Software Foundation. *Errors and Exceptions* (manejo de excepciones). <https://docs.python.org/es/3/tutorial/errors.html>
- Python Software Foundation. *Sorting Techniques* (sorted y key). <https://docs.python.org/es/3/howto/sorting.html>
- Python Software Foundation. *Built-in Functions* (max, min, sum, sorted). <https://docs.python.org/es/3/library/functions.html>

6. Enlaces de la entrega

- **Repositorio GitHub:** [pegar aquí el enlace público del repositorio]
- **Video explicativo (10-15 min):** [pegar aquí el enlace público del video]